

HPCプログラミングセミナー

GPUプログラミング入門（OpenACC編）

第3講：OpenACC応用

高度情報科学技術研究機構

Research Organization for Information Science and Technology (RIST)

質問は随時受け付けます

E-mail (hpc-seminar@rist.or.jp) もご利用ください

この資料は、HPCプログラミングセミナー「アクセラレータ入門」として実施していたものと同じ内容です
OpenACCを用いたGPUプログラミングの基礎を解説するという資料の主旨を明確にするため、タイトルを変更しました

本資料の教育目的等での転載、複製及び再配布をご希望される場合は資料末尾の記述を参照ください

本講の目的

- 第1, 2講の内容を元とした、より実践的なGPUの活用に関役立つトピックを紹介します
 - ▶ 主としてNVIDIA HPC SDKの利用を前提に説明

自学自習のために

■ サンプルコードも用意（ご自由にお使いください）

- ▶ OpenACCを中心に、OpenMP Target, Kokkosのサンプルを含む
- ▶ HPCIポータルサイトから入手可能
 - ▶ https://www.hpci-office.jp/events/seminars/seminar_texts

概要

1. ユニファイドメモリ / マネージドメモリ
2. プロファイラの実践的な利用
3. CPU・GPUの非同期処理
4. OpenACCとCUDAのハイブリッド利用
5. 複数GPUの利用
6. Tipsの紹介

1. ユニファイドメモリ / マネージドメモリ

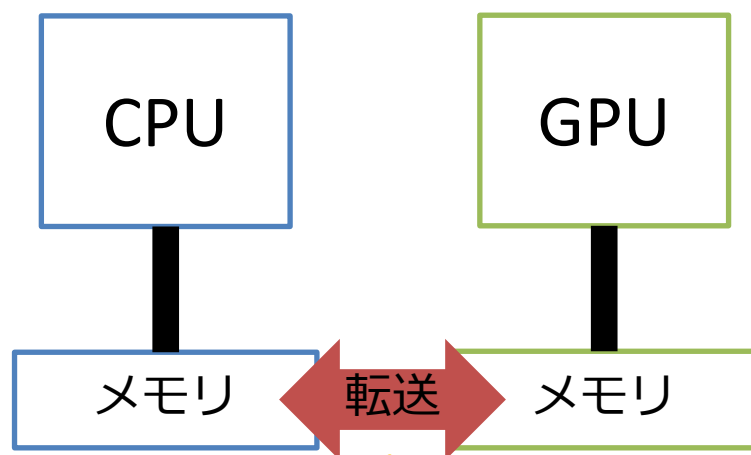
ユニファイドメモリ / マネージドメモリ

- Pascal世代GPU以降で実装されている、ホストとデバイスのメモリを一元的に管理する機能

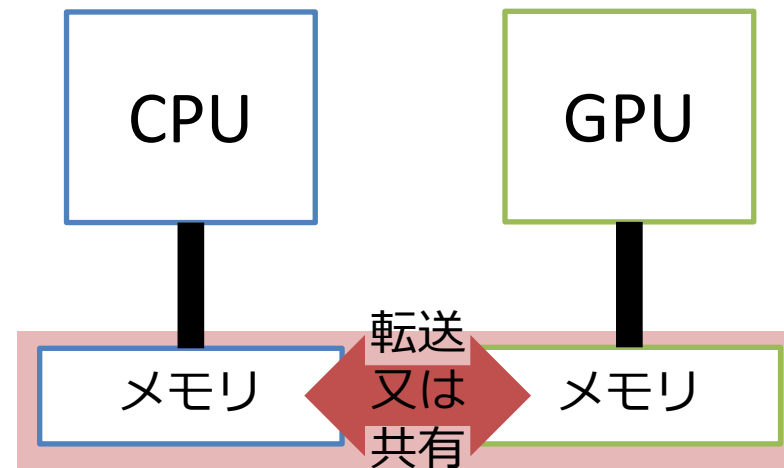
▶ 最新のGPU（G200等）では実装が異なるが、機能的には同じもの

非ユニファイド（セパレート）メモリ

ユニファイドメモリ / マネージドメモリ



コンパイラ任せ又はdata構文



自動で転送又は共有される

※資料内では
両者を明確に
は区別しない

メモリが別れていることを意識すること無く記述できる

OpenACCとユニファイドメモリ (1/2)

- ホストデバイス間のデータ管理がシステム任せになり、結果OpenACCと親和性が高い
 - ▶ ドライバが自動的にデータ転送。ユーザがdata構文を書く必要がない
 - ▶ 複数GPU利用にてMPIライブラリをコードの変更無しで利用できる（「5.複数GPUの利用」を参照）
 - ▶ deep copy問題の対処策として有効（発展）
- 注意すること
 - ▶ 動的に確保するメモリのみ対応
 - ▶ CUDA対応MPI（後述）が利用できない
 - ▶ 非同期処理（有効な最適化手段）が利用できない
 - ▶ 明示的なデータ転送よりも少し遅い。
 - ▶ [参考]株式会社Metro, 「GPU高速化事例 vol.1」, <https://www.metro.co.jp/casestudy/>
 - ▶ 冗長な転送を完全に排除できるとは限らない

OpenACCとユニファイドメモリ (2/2)

- コンパイル・オプションの設定のみで利用可能（具体的な指定方法はドキュメントで確認する必要あり）
- 例: `nvc -acc -Minfo=accel -gpu=cc80,cuda12.3,managed jacobi.c -o jacobi.out`
 - ▶ 備考1: `-gpu=pinned`と`-gpu=managed`は併用不可
 - ▶ 備考2: オプション指定はコンパイラのバージョンに依存
 - ▶ `-gpu=managed` (NVIDIA HPC SDK 23.11)
 - ▶ `-gpu=mem:managed` (NVIDIA HPC SDK 24.9, 25.1)
 - ▶ cf. `-gpu=mem:unified:nomanagedalloc` → GH200での「ユニファイドメモリ」
- `nvaccelinfo`の出力結果（後述）から利用の可否を確認可能

第2講ヤコビ法の最適化への適用 (A100)

■ タイマー挿入で経過時間の比較 (NVIDIA A100)

GPU版共通オプション:
-gpu=cc80,cuda12.3

- ▶ 問題規模: 8192*8192, 反復は100回で強制打ち切り(未収束)
- ▶ gpu=managedでデータ転送最適化版と同程度の性能

Type	# of CPU cores	# of GPUs	Elapsed time of a main loop(sec.)	Speedup
CPU (acc=multicore)	36	-	1.97	1.0 (baseline)
GPU (acc=gpu, gpu=pinned)	1	1	10.6	0.19
GPU (acc=gpu, gpu=pinned, データ転送最適化版)	1	1	0.299	6.6
GPU (acc=gpu, gpu=managed, データ転送最適化未適用)	1	1	0.405	4.9

第2講ヤコビ法の最適化への適用 (v100)

■ cf. タイマで経過時間の比較 (NVIDIA Tesla V100)

GPU版共通オプション:
-gpu=cc70,cuda11.4

▶ 問題規模: 8192*8192

▶ 注意: A100の計測とコードと反復回数が異なる。

▶ gpu=managedが性能にpositiveなimpact

Type	# of CPU cores	# of GPUs	Elapsed time of a main loop(sec.)	Speedup
CPU (acc=multicore)	20	-	30.5	1.0 (baseline)
GPU (acc=gpu, gpu=pinned)	1	1	171.6	0.18
GPU (acc=gpu, gpu=pinned, データ転送最適化版)	1	1	3.5	8.7
GPU (acc=gpu, gpu=managed, データ転送最適化未適用)	1	1	3.7	8.2

HMM対応のGPUアーキテクチャ (1/3)

■ HMM (Heterogeneous Memory Management)

- ▶ <https://www.kernel.org/doc/html/latest/mm/hmm.html>
 - ▶ (雑に言えば) CPUとGPUとの間でメモリ(アドレス空間)を共有する仕組み
- ▶ 全てのGPUアーキテクチャがHMMに対応している訳では無い
 - ▶ ただし, 対応するGPUは増えている。
- ▶ OpenACC: `-gpu= mem:unified:nomanagedalloc`で使われるユニファイドメモリと関連
 - ▶ 参考: マネージドメモリ (“-gpu=managed”) のユニファイドメモリとの違い
 - ▶ <https://forums.developer.nvidia.com/t/openacc-confusion-about-managed-and-uniform-memory/283430>

HMM対応のGPUアーキテクチャ (2/3)

■ 例1: NVIDIA GraceHopper Superchip (GH200)

- ▶ Miyabi(JCAHPC (筑波大/東大))で採用

- ▶ <https://www.jcahpc.jp/supercomputer/miyabi.html>

- ▶ HPCIでの資源提供もあり

- ▶ OpenACC: コンパイルオプションで

- gpu= mem:unified:nomanagedallocを指定

- ▶ 参考情報

- ▶ <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper>

- ▶ <https://developer.nvidia.com/blog/simplifyinggpu-programming-for-hpc-with-the-nvidia-grace-hopper-superchip/>

- ▶ “Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip”, arXiv:2408.11556

HMM対応のGPUアーキテクチャ (3/3)

■例2: AMD Instinct™ MI300A

- ▶ El Capitan (LLNL, US)で採用

- ▶ <https://asc.llnl.gov/exascale/el-capitan>

- ▶ 参考情報

- ▶ <https://www.amd.com/en/technologies/cdna.html>

- ▶ AMD CDNA™ 3のwhite paperを参照

- ▶ “Porting HPC Applications to AMD Instinct™ MI300A Using Unified Memory and OpenMP”, arXiv: 2405.00436

2. プロファイラの実践的な利用

NVIDIAプロファイラの概要

■ NVIDIA Nsight Systems (nsys, nsys-ui)

- ▶ 包括的にコスト分布、計算特性や指標を計測することが中心
- ▶ ボトルネックの特定向き
- ▶ CPUの性能計測も備え、CPU-GPUを包括的に性能を調査できる

■ NVIDIA Nsight Compute (ncu, ncu-ui)

- ▶ GPUカーネル単位で詳細に分析するツール
 - ▶ キャッシュミス、メモリスループットなど
- ▶ Nsight Systemsの分析をさらに詳しく調査する場合に活用できる

■ 本講ではnsysのコマンドラインでの利用を中心にまとめる

- ▶ スパコンのリモート利用を想定、GUI利用はローカル環境にて

nsys 基本的な利用方法

■ Basic style

▶ nsys [command_switch] [options] [application] [application options]

■ command_switchで機能の切り替え

▶ 種類が多い: 機能豊富なプロファイラ

▶ 性能情報取得で最も使うのはprofile

▶ アプリケーションの最適化において十分な機能を含む

nsys profile [options] [application] [application options]

nsys profile 推奨利用方法

■ `nsys profile -t cuda,nvtx,openacc --stats=true`

- ▶ profile: 性能情報取得を指定

- ▶ -t: traceする項目を指定

 - ▶ cuda: CUDA関数、CUDA API

 - ▶ openacc: OpenACC API

 - ▶ nvtx: NVIDIA Tool Extension（後述）利用の有無

- ▶ --stats: 基本的な性能情報のサマリの出力の指定、true指定で標準出力に出力

■ 標準出力の他に.nsys-rep形式が出力され、時系列データをGUI（nsys-ui）に取込み可能

NVTX (NVIDIA Tools Extensions)

- ソースコードに計測区間指定（アノテーション）を付けられるライブラリ
 - ▶ 区間指定以外の機能も有するがここでは省略 詳しくは公式資料参照
- プロファイラの標準はCUDA APIや関数などが出力の単位
 - ▶ コストの概要を把握するには不向きな場合も
 - ▶ 同じ名前の関数がコード内で分散して配置されている等
- ユーザの区間指定のメリット
 - ▶ アプリケーションの構造に合わせることができる
- ちょっと便利な付加機能もあるので使いこなすと便利
 - ▶ ex. 区間別に色分けしてGUIに表示

NVTX 区間指定API使用例 [c]

```
#include <nvtx3/nvToolsExt.h>
```

```
nvtxRangeId_t id1 = nvtxRangeStartA("Total");  
init();
```

```
nvtxRangeId_t id2 = nvtxRangeStartA("while");  
while (etime < sim_time)  
{
```

```
    nvtxRangeId_t id3 = nvtxRangeStartA("step_1");  
    step_1();  
    nvtxRangeEnd(id3);
```

```
    nvtxRangeId_t id4 = nvtxRangeStartA("step_2");  
    step_2();  
    nvtxRangeEnd(id4);
```

```
}  
nvtxRangeEnd(id2);
```

```
finalize();  
nvtxRangeEnd(id1);
```

詳しい仕様: <https://github.com/NVIDIA/NVTX>

■ 4区間を指定

- ▶ Total, while, step_1, step_2
- ▶ 階層的に指定可能

■ ソースコード

- ▶ #include “nvtx3/nvToolsExt.h”を記述

■ コンパイル

- ▶ ヘッダーファイルをインクルード
 - ▶ コンパイラ側で自動的に解決
 - ▶ 失敗する場合は次を設定

-I\$NVHPC_ROOT/cuda/[CUDA version]/include

■ nsys実行時にnvtxをtオプションにて指定する

- ▶ nsys profile -t nvtx

NVTX 区間指定API使用例 [Fortran]

詳しい仕様: <https://github.com/NVIDIA/NVTX>

```
use nvtx

call nvtxStartRange("Total")
call init()

call nvtxStartRange("while")
do while (etime < sim_time)

    call nvtxStartRange("step_1")
    call step_1
    call nvtxEndRange

    call nvtxStartRange("step_2")
    call step_2
    call nvtxEndRange

enddo
call nvtxEndRange

call finalize()
call nvtxEndRange
```

■ 4区間を指定

- ▶ Total, while, step_1, step_2
- ▶ 階層的に指定可能

■ ソースコード

- ▶ use nvtxを記述

■ リンク

- ▶ -lnvhpcwrapnvtxを設定

■ nsys実行時にnvtxをtオプションにて指定する

- ▶ nsys profile -t nvtx

NVTX区間指定適用例: 第2講のヤコビ法

```
020: #include <nvtx3/nvToolsExt.h>
...
084:  nvtxRangeId_t id0 = nvtxRangeStartA("data");
086:  #pragma acc data ...
087:  {
    /* initialize phio */
098:  nvtxRangeId_t id1 = nvtxRangeStartA("rept");
100:  iconv = 0;
101:  for ( itr = 1; itr <= MAXITR; ++itr) {
104:  nvtxRangeId_t id2 = nvtxRangeStartA("update");
106:    nrmsq = 0.0;
107:    #pragma acc data ...
108:    #pragma acc kernels
109:    #pragma acc loop ... reduction(max:nrmsq)
110:    for (int ix = 1; ix < NX-1; ++ix) {
111:      for (int iy = 1; iy < NY-1; ++iy) {
112:        phio[IDX(ix,iy)] = ...
115:        nrmsq = ...
116:      }
117:    }
119:  nvtxRangeEnd(id2);
133:  nvtxRangeId_t id3 = nvtxRangeStartA("swap");
135:  #pragma acc data present(phie[0:NX*NY])
136:  #pragma acc kernels
137:  #pragma acc loop independent collapse(2)
138:  for ( int ix = 0; ix < NX; ++ix ) {
139:    for ( int iy = 0; iy < NY; ++iy ) {
140:      phie[IDX(ix,iy)] = phio[IDX(ix,iy)];
141:    }
142:  }
```

```
144:  nvtxRangeEnd(id3);
147:  /* check convergence */
151:  } /* itr */
153:  nvtxRangeEnd(id1);
155:  } /* acc data */
157:  nvtxRangeEnd(id0);
```

A diagram enclosed in an orange border showing the sequence of NVTX ranges. It consists of four horizontal lines, each starting with a vertical bar and followed by a range name: 'data' (blue), '--rept' (red), '--update' (green), and '--swap' (green). The labels are aligned to the right of the vertical bars.

--stats=true テキスト表示

NVIDIA A100での計測例

■全体をおおまかに捉える: ボトルネックの特定に良い

NVTX 区間指定 出力：時間

[3/8] Executing 'nvtx_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
44.0	298598701	1	298598701.0	298598701.0	298598701	298598701	0.0	StartEnd	data
28.0	190096331	1	190096331.0	190096331.0	190096331	190096331	0.0	StartEnd	rept
18.2	123545013	100	1235450.1	1233545.0	1224244	1424269	19476.3	StartEnd	update
9.8	66151285	100	661512.9	661422.5	658517	665167	1250.0	StartEnd	swap

CUDA関数 (-t cuda) 出力：時間

[5/8] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
58.2	106457721	100	1064577.2	1064088.0	1056424	1072072	3352.2	_6main_c_main_111_gpu
35.2	64451766	100	644517.7	644548.5	642085	647237	1081.3	_6main_c_main_139_gpu
6.5	11978523	100	119785.2	119713.0	117442	122017	1092.6	_6main_c_main_111_gpu_red

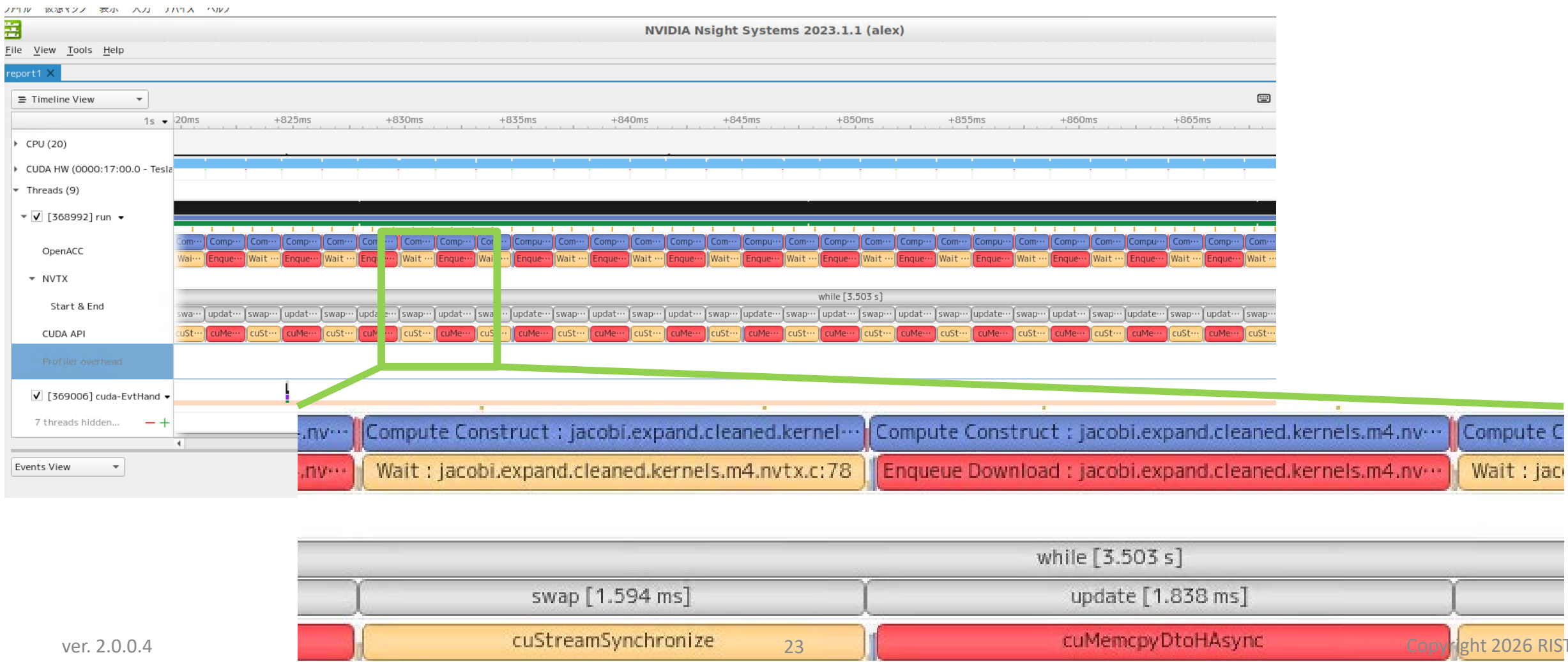
CUDA メモリ転送 (-t cuda) 出力：時間

[6/8] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
66.3	42512141	2	21256070.5	21256070.5	21236550	21275591	27606.2	[CUDA memcpy Host-to-Device]
33.6	21518822	101	213057.6	1504.0	1440	21366663	2125911.0	[CUDA memcpy Device-to-Host]
0.2	105666	100	1056.7	1056.0	1023	1440	41.7	[CUDA memset]

Nsight Systems GUI timeline view

■ 時間情報（開始、終了、区間）をGUIで並べて表示



3. CPU・GPUの非同期処理

非同期処理でCPUとGPUのリソースを有効活用

CPUとGPUが同期実行例

```
program main
  implicit none
```

ここまで解説した、GPUへ
「オフロード」するやり方
→無駄はないか？

```
  a(i) = 1; b(i) = 1*10
end do
```

```
!$acc kernels loop
do i = 1, n
  a(i) = a(i) + 2.0
  b(i) = b(i) * 2.0
end do
!$acc end kernels
```

```
  print *, a(:), b(:)
end program
```

CPU

GPU

CPU実
行

GPU実
行

CPU実
行

CPUとGPUが非同期実行例

```
program main
  implicit none
```

CPU・GPUに処理を振り分
けて効率を上げることが
出来る？

```
  a(i) = 1; b(i) = 1*10
end do
```

```
!$acc kernels loop async
do i = 1, n
  a(i) = a(i) + 2.0
end do
!$acc end kernels
```

```
do i = 1, n
  b(i) = b(i) * 2.0
end do
!$acc wait
```

```
  print *, a(:), b(:)
end program
```

CPU

GPU

CPU実
行

GPU実
行

同期実行の例

■ GPU実行とCPU実行が逐次に処理

- ▶ 先行の終了を待ってから次に

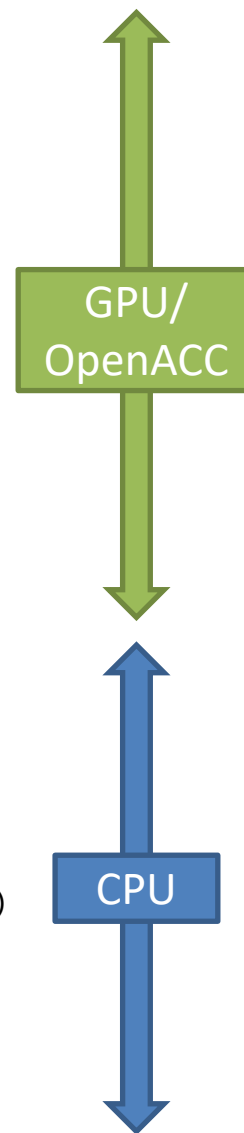
■ CPU・GPUのどちらか一方のリソースのみ利用している状態

```
call nvtxStartRange("total")
!$acc data copyin(a,b) copyout(c)
```

```
call nvtxStartRange("OpenACC")
!$acc kernels
do j=1,n
!$acc loop seq
  do i=1,n
    tmp = 0.0
!$acc loop seq
    do k=1,n
      tmp = tmp + a(i,k) * b(k,j)
    enddo
    c(i,j) = tmp
  enddo
enddo
!$acc end kernels
call nvtxEndRange
```

```
call nvtxStartRange("Serial")
do j=1,n2
  do i=1,n2
    tmp = 0.0
    do k=1,n2
      tmp = tmp + a2(i,k) * b2(k,j)
    enddo
    c2(i,j) = tmp
  enddo
enddo
call nvtxEndRange
```

```
!$acc end data
call nvtxEndRange
```



非同期実行の例

■ 「async + wait」の組み合わせ

- ▶ async節

- ▶ wait構文

■ GPU実行とCPU実行が並行に処理

- ▶ 両方のリソースを並行して利用

■ 処理間の依存性の有無を確認すること

- ▶ 結果不正を起こす可能性あり

```
call nvtxStartRange("total")
!$acc data copyin(a,b) copyout(c)
```

```
call nvtxStartRange("OpenACC")
!$acc kernels async(1)
do j=1,n
!$acc loop seq
  do i=1,n
    tmp = 0.0
!$acc loop seq
    do k=1,n
      tmp = tmp + a(i,k) * b(k,j)
    enddo
    c(i,j) = tmp
  enddo
enddo
!$acc end kernels
```

OpenACCデフォルトはここで待つ

```
call nvtxStartRange("Serial")
do j=1,n2
  do i=1,n2
    tmp = 0.0
    do k=1,n2
      tmp = tmp + a2(i,k) * b2(k,j)
    enddo
    c2(i,j) = tmp
  enddo
enddo
call nvtxEndRange
!$acc wait(1)
call nvtxEndRange
```

```
!$acc end data
call nvtxEndRange
```

GPU/
OpenACC

CPU

Async指定してる
のですぐに戻る

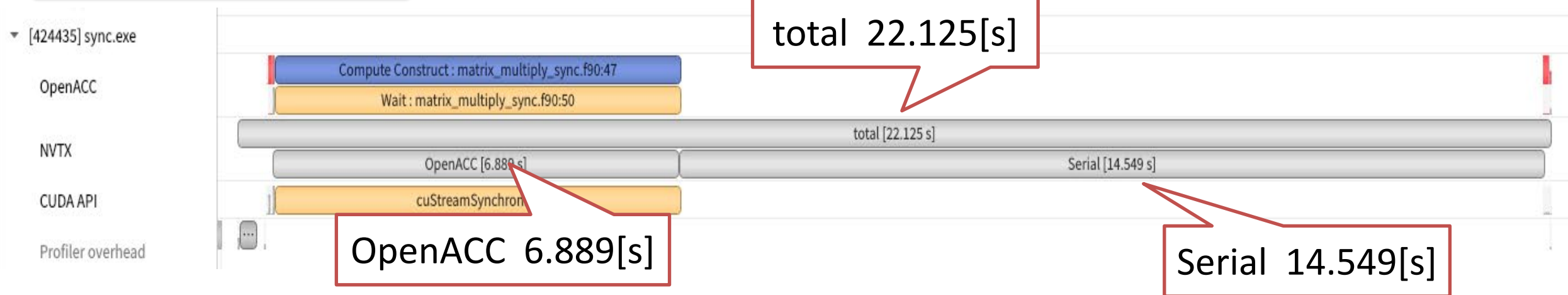
すぐに戻っている
のでGPUを待たず
に開始（コード上
の対応とは違う）。

同期が取られて整理される

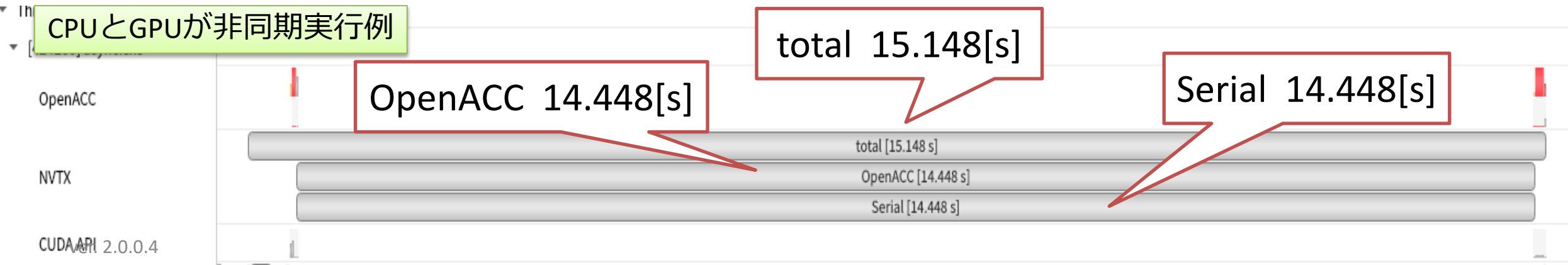
プロファイラ（nsys-ui with NVTX）による分析

- 同期と非同期それぞれのタイムライン
 - ▶ totalの時間が削減されていることが分かる

CPUとGPUが同期実行例



CPUとGPUが非同期実行例



4. OpenACCとCUDAのハイブリッド利用

概要

- デバイスデータの受け渡し（OpenACC↔CUDA）が必要
 - ▶ 同じデバイスデータを共有可能
 - ▶ ホストを経由する必要なし
- OpenACCで扱ったデバイス上のデータをCUDAで記述した関数に渡す
 - ▶ host_data構文を利用する (use_device節で指定)
 - ▶ CUDAで実装されたライブラリ (例: cuBLAS)でも使える方法
 - ▶ <https://docs.nvidia.com/cuda/cublas/>
 - ▶ CUDA対応MPI利用でも使える方法 (「複数GPUの利用」を参照)
- CUDAの記述については本講では扱いません
 - ▶ CUDA C++ Programming Guide等を参照ください
 - ▶ <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

コンパイル

- CUDAカーネルを含むファイルとOpenACCを含むファイルとは別に用意
 - ▶ OpenACC、CUDAでそれぞれ個別にコンパイルした後リンクする
 - ▶ コンパイラとオプションは個別に指定
 - ▶ CUDA C: nvcc OpenACC C: nvc
 - ▶ CUDA Fortran/OpenACC Fortranは共にnvfortran利用でオプションで区別
- CUDA ↔ OpenACCの間はwrapper関数を用意してつなぐ
 - ▶ OpenACC実装、CUDA実装それぞれをコードしてなるべく汚さない

OpenACCからCUDA [c]

openacc_main.c

```
extern void vecadd(int, float*, float*, float*);

int main()
{
    float *a, *b, *c;
    int i, n;
    n = 1024;

    a = (float*)malloc(n*sizeof(float));
    b = (float*)malloc(n*sizeof(float));
    c = (float*)malloc(n*sizeof(float));

    #pragma acc data create(a[0:n], b[0:n]) copyout(c[0:n])
    {
        #pragma acc kernels
        for( i = 0; i < n; i++)
        {
            a[i] = (float)i;
            b[i] = (float)i*2.0f;
        }
        #pragma acc host_data use_device(a, b, c)
        {
            vecadd(n, a, b, c);
        }

        for( i = 0; i < 10; i++)    fprintf(stdout, "c[%d] = %f\n", i, c[i]);

        return 0;
    }
```

a,b,c はデバイスポインタ
で渡すことを指示

CUDA関数を含むwrapper
関数で呼び出す

vecadd.cu

```
__global__ void vecadd_kernel(int n, float *a, float *b, float *c)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    if ( i < n ) c[i] = a[i] + b[i];
}

extern "C" void vecadd(int n, float *a, float *b, float *c)
{
    dim3 griddim, blockdim;

    blockdim = dim3(128, 1, 1);
    griddim = dim3(n/blockdim.x, 1, 1);

    vecadd_kernel<<<griddim, blockdim>>>(n, a, b, c);
}
```

コンパイル例

```
$ nvc -acc -Minfo -O2 -c openacc_main.c ! OpenACC
```

```
$ nvcc -c -O2 vecadd.cu ! Wrapper + CUDA関数
```

```
$ nvc -acc -Mcuda openacc_main.o vecadd.o ! リンク
```


OpenACCからCUDA [Fortran]

openacc_main.f90

```
program main
  use vecadd_module
  integer(4), parameter :: n = 2**20
  real(4), allocatable, dimension(:) :: a, b, c

  allocate(a(n))
  allocate(b(n))
  allocate(c(n))

  !$acc data create(a,b) copyout(c)
  !$acc kernels
  do i = 1, n
    a(i) = i
    b(i) = i * 2.0
  end do
  !$acc end kernels

  !$acc host_data use_device(a,b,c)
  call vecadd_wrapper(n, a, b, c)
  !$acc end host_data

  !$acc end data

  print '(1x, i10, F15.0)', (i, c(i), i=n-10,n)
end program
```

a,b,c はデバイスポインタで渡すことを指示

CUDAカーネルを含むwrapperで呼び出す

vecadd.cuf

```
module vecadd_module
  contains
  attributes(global) subroutine vecadd_kernel (n, a, b, c)
    real(4), device :: a(:), b(:), c(:)
    integer(4), value :: n, i

    i = (blockIdx%x-1)*blockDim%x + threadIdx%x
    if (i<=n) c(i) = a(i) + b(i)
  end subroutine

  subroutine vecadd_wrapper(n, a, b, c)
    use cudafor
    real, device :: a(:), b(:), c(:)
    integer :: n
    type(dim3) :: blocks, threads

    blocks = dim3(n/128,1,1); threads = dim3(128, 1, 1)
    call vecadd_kernel<<<blocks, threads>>>(n, a, b, c)
  end subroutine
end module vecadd_module
```

コンパイル例

```
$ nvfortran -acc -O2 -Minfo -c openacc_main.f90 ! OpenACC

$ nvfortran -Mcuda -O2 -Minfo -c vecadd.cuf
! Wrapper + CUDA関数

$ nvfortran -acc -Mcuda openacc_main.o vecadd.o ! リンク
```

実験: DGEMM in cuBLASの利用 (1/2)

```
#pragma acc data copyin(mata[0:NSIZE*NSIZE], matb[0:NSIZE*NSIZE]) copy(matc[0:NSIZE*NSIZE])
{
    icon = 0;
    for ( int it = 0; it < NITR; ++it) {
        mmp_simple_acc_gv(NSIZE, matc, mata, matb);
        icon += dummy(it, NSIZE, matc);
    }
} /* acc data */

/* ... */

#pragma acc data copyin(mata[0:NSIZE*NSIZE], matb[0:NSIZE*NSIZE]) copy(matc_trans[0:NSIZE*NSIZE])
{
    icon = 0;
    for ( int it = 0; it < NITR; ++it) {
        #pragma acc host_data use_device(mata, matb, matc_trans)
        {
            cublasDgemm(cublas_handle, CUBLAS_OP_T, CUBLAS_OP_T, NSIZE, NSIZE, NSIZE,
                        &one, mata, NSIZE, matb, NSIZE, &zero, matc_trans, NSIZE);
        } /* acc host_data */

        const cudaError_t error = cudaDeviceSynchronize();
        if ( error != cudaSuccess) {
            exit(1);
        }
        icon += dummy(it, NSIZE, matc);
    }
} /* acc data */
```

OpenACCで素朴に実装したDGEMM

```
#pragma acc loop gang independent
for ( int i = 0; i < ns; ++i ) {
    #pragma acc loop vector independent
    for ( int j = 0; j < ns; ++j ) {
        double mctmp = 0.0;
        #pragma acc loop seq
        for ( int k = 0; k < ns; ++k ) {
            mctmp += ma[k + i*ns]*mb[j + k*ns];
        }
        mc[j + ns*i] += mctmp;
    }
}
```

OpenACCのdata節でデータ転送したデータをcuBLAS側でアクセスできるように指定

cuBLASのDGEMM

cuBLAS;
<https://docs.nvidia.com/cuda/cublas/index.html>

実験: DGEMM in cuBLASの利用 (2/2)

■ 計測例: NVIDIA GPU H100 in TSUBAME 4.0 (東京科学大)

- ▶ NVIDIA HPC SDK 24.2
- ▶ 倍精度浮動小数点, 正方行列
- ▶ 行列次元が大きいときはcuBLASを使うほうが有利

```
コンパイル: nvc -g -c11 -O3 -acc=gpu -gpu=cc90,sm_90,cuda12.3
             -tp=zen3 -Minfo=accel -cuda -c ...
リンク: nvc -g -c11 -acc=gpu -gpu=cc90,sm_90,cuda12.3
          -tp=zen3 -cuda -cudalib=cublas ...
```

- ▶ flop/s(はラフに評価: $2 \times N^3 \times \text{計測回数} / \text{経過時間}$), with N: 行列次元

100 x 100行列

	計測回数	経過時間 (s)	Gflop/s
mmp_simple_acc_gv	20000	0.239	167.40
cublasDgemm	20000	0.280	142.60

1000 x 1000行列

	計測回数	経過時間 (s)	Gflop/s
mmp_simple_acc_gv	1000	0.628	3183.3
cublasDgemm	1000	0.061	32719

5. 複数GPUの利用

複数GPU+OpenACC

- 「MPIを使った領域分割 + OpenACC」を想定して, 複数GPUを利用する例を紹介
 - ▶ タスク並列は本講では無し
- MPI並列化済みのコードを推奨。
 - ▶ ノード内限定ならOpenMPの利用も可能だが記述が複雑化
 - ▶ MPI + OpenACCはノード内/間両方に対応し、汎用性高い
 - ▶ MPI並列の記述に少ない変更（指示文挿入）で実装可能
- GPU特有の記述2つを紹介
 - ▶ 「デバイス番号の指定」「MPIへのデバイスメモリの渡し方」
- お薦めの資料：GitHub jirikraus <https://github.com/jirikraus>
 - ▶ 学習用サンプルと説明スライドが入手可能

デバイス番号の指定 (1/2)

- GPUそれぞれに0, 1, 2,,と各ノード内でふられたローカルな番号
- デバイス番号↔MPIプロセスの対応はユーザが行う
 - ▶ 1MPIプロセス↔1GPUの対応が基本
- OpenACC APIを用いて設定
 - ▶ C: #include <openacc.h> Fortran: use openacc 必要

C

```
#include <openacc.h>

int ngpus=acc_get_num_devices(acc_device_nvidia);

int devicenum=rank%ngpus;

acc_set_device_num(devicenum, acc_device_nvidia);
```

Fortran

```
use openacc

ngpus=acc_get_num_devices(acc_device_nvidia)

devicenum=mod(rank, ngpus)

call acc_set_device_num(devicenum, acc_device_nvidia)
```

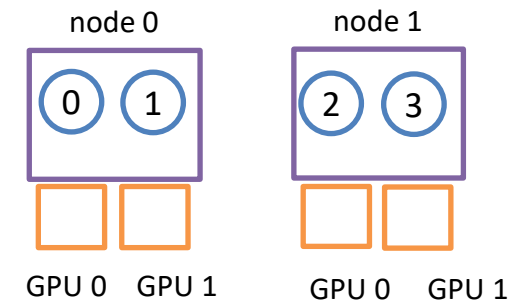
デバイス番号の指定 (2/2)

■ 例: 2ノード, 2 GPUs/ノードの計算機

▶ ノードあたり2MPIプロセスで実行

▶ MPIランク: node 0で0, 1, node1で2, 3と想定

▶ 常にこうなるとは限らないことに注意 (計算機環境と実行時設定に依存)



C

```
#include <openacc.h>
```

ngpus=2

```
int ngpus=acc_get_num_devices(acc_device_nvidia);
```

```
int devicenum=rank%ngpus;
```

```
acc_set_device_num(devicenum, acc_device_nvidia);
```

ランク0: devicenum = 0
ランク1: devicenum = 1
ランク2: devicenum = 0
ランク3: devicenum = 1

Fortran

```
use openacc
```

ngpus=2

```
ngpus=acc_get_num_devices(acc_device_nvidia)
```

```
devicenum=mod(rank, ngpus)
```

```
call acc_set_device_num(devicenum, acc_device_nvidia)
```

ランク0: devicenum = 0
ランク1: devicenum = 1
ランク2: devicenum = 0
ランク3: devicenum = 1

ヤコビ法 (MPI版, データ転送最適化無)

CPU

GPU

```
56
57 while ( res > res_c && it < it_max ) {
58     double res = 0.0;
59 #pragma acc data copyin(A[0:n*m]) copyout(Anew[0:n*m])
60 #pragma acc kernels
61 #pragma acc loop independent reduction(max:res)
62 for( int j = 1; j < n-1; j++) {
63     for( int i = 1; i < m-1; i++ ) {
64         Anew[IDX(j, i)] = 0.25 * ( A[IDX(j, i+1)] + A[IDX(j, i-1)]
65                                   + A[IDX(j-1, i)] + A[IDX(j+1, i)] );
66         res = fmax( res, fabs(Anew[IDX(j, i)] - A[IDX(j, i)]) );
67     }
68 }
69
70 #pragma acc data copyin(Anew[0:n*m]) copyout(A[0:n*m])
71 #pragma acc kernels
72 #pragma acc loop independent
73 for( int j = 1; j < n-1; j++) {
74     for( int i = 1; i < m-1; i++ ) {
75         A[IDX(j, i)] = Anew[IDX(j, i)];
76     }
77 }
78
79 pack_halodata( buf, A, , , , );
80 MPI_Sendrecv( buf[jstart], , , , );
81 unpack_halodata( buf, A, , , , );
82 it++;
83 }
ver. 2.0}0.4
```

A

Anew

Anew

ホスト (CPU) のAは最新の
ものに更新されている

問題なく動作する

A

ヤコビ法 (MPI版, データ転送最適化有)

CPU

GPU

```
56 #pragma acc data copy(A[0:n*m]) create(Anew[0:n*m])
57 while ( res > res_c && it < it_max ) {
58     double res = 0.0;
59 #pragma acc data present(A[0:n*m], Anew[0:n*m])
60 #pragma acc kernels
61 #pragma acc loop independent reduction(max:res)
62     for( int j = 1; j < n-1; j++) {
63         for( int i = 1; i < m-1; i++ ) {
64             Anew[IDX(j, i)] = 0.25 * ( A[IDX(j, i+1)] + A[IDX(j, i-1)]
65                                     + A[IDX(j-1, i)] + A[IDX(j+1, i)] );
66             res = fmax( res, fabs(Anew[IDX(j, i)] - A[IDX(j, i)]));
67         }
68     }
69
70 #pragma acc data present(Anew[0:n*m], A[0:n*m])
71 #pragma acc kernels
72 #pragma acc loop independent
73     for( int j = 1; j < n-1; j++) {
74         for( int i = 1; i < m-1; i++ ) {
75             A[IDX(j, i)] = Anew[IDX(j, i)];
76         }
77     }
78
79     pack_halodata( buf, A, , , , );
80     MPI_Sendrecv( buf[jstart], , , , );
81     unpack_halodata( buf, A, , , , );
82     it++;
83 }
```

ホストのAはwhileループを抜けるまでは更新無し
→Aの初期値がMPIライブラリに渡されてしまう
→デバイスのAは袖が更新されない
→結果不正

※ユニファイドメモリに任せる事もできる
その場合は本スライドの記述で動作する

ユニファイドメモリを使わない、MPIヘ
デバイスメモリを陽に渡す方法を2つ紹介

A

A

ヤコビ法 (MPI版, データ転送最適化有)

CPU

GPU

```
56 #pragma acc data copy(A[0:n*m]) create(Anew[0:n*m])
57 while ( res > res_c && it < it_max ) {
58     double res = 0.0;
59 #pragma acc data present(A[0:n*m], Anew[0:n*m])
60 #pragma acc kernels
61 #pragma acc loop independent reduction(max:res)
62     for( int j = 1; j < n-1; j++) {
63         for( int i = 1; i < m-1; i++ ) {
64             Anew[IDX(j, i)] = 0.25 * ( A[IDX(j, i+1)] + A[IDX(j, i-1)]
65                                     + A[IDX(j-1, i)] + A[IDX(j+1, i)] );
66             res = fmax( res, fabs(Anew[IDX(j, i)] - A[IDX(j, i)]));
67         }
68     }
69
70 #pragma acc data present(Anew[0:n*m], A[0:n*m])
71 #pragma acc kernels
72 #pragma acc loop independent
73     for( int j = 1; j < n-1; j++) {
74         for( int i = 1; i < m-1; i++ ) {
75             A[IDX(j, i)] = Anew[IDX(j, i)];
76         }
77     }
78     pack_halodata( buf, A, , , , );
79 #pragma acc update host(buf)
80     MPI_Sendrecv( buf[jstart], , , , );
81 #pragma acc update device(buf)
81     unpack_halodata( buf, A, , , , );
82     it++; }
```

方法 1

MPIライブラリの前後でupdate構文
袖bufのみホストデバイス転送

MPIライブラリにbufのホストメモリの
ポインタを指定して渡す→正常に動作



ヤコビ法 (MPI版, データ転送最適化有)

CPU

GPU

```
56 #pragma acc data copy(A[0:n*m]) create(Anew[0:n*m])
```

```
57 while ( res > res_c && it < it_max ) {
```

```
58     double res = 0.0;
```

```
59 #pragma acc data present(A[0:n*m], Anew[0:n*m])
```

```
60 #pragma acc kernels
```

```
61 #pragma acc loop independent reduction(max:res)
```

```
62     for( int j = 1; j < n-1; j++) {
```

```
63         for( int i = 1; i < m-1; i++ ) {
```

```
64             Anew[IDX(j, i)] = 0.25 * ( A[IDX(j, i+1)] + A[IDX(j, i-1)]
```

```
65                                     + A[IDX(j-1, i)] + A[IDX(j+1, i)]);
```

```
66             res = fmax( res, fabs(Anew[IDX(j, i)] - A[IDX(j, i)]));
```

```
67         }
```

```
68     }
```

```
69
```

```
70 #pragma acc data present(Anew[0:n*m], A[0:n*m])
```

```
71 #pragma acc kernels
```

```
72 #pragma acc loop independent
```

```
73     for( int j = 1; j < n-1; j++) {
```

```
74         for( int i = 1; i < m-1; i++ ) {
```

```
75             A[IDX(j, i)] = Anew[IDX(j, i)];
```

```
76         }
```

```
77     }
```

```
78     pack_halodata( buf, A, , , , );
```

```
79 #pragma acc host_data use_device(buf)
```

```
80     MPI_Sendrecv( buf[jstart], , , , );
```

```
81     unpack_halodata( buf, A, , , , );
```

```
82     it++;
```

```
83 ver. 2.0.4
```

方法2

CUDA対応MPIライブラリ
(デバイスポインタを直接渡せる)
+ host_data構文

bufのホスト↔デバイス
転送がCUDA対応MPI内
部で発生しているはず
(後述のGPUDirect
RDMA利用時は別)

MPIライブラリにbufのデバイスメモリ
のポインタを指定して渡す

A

A

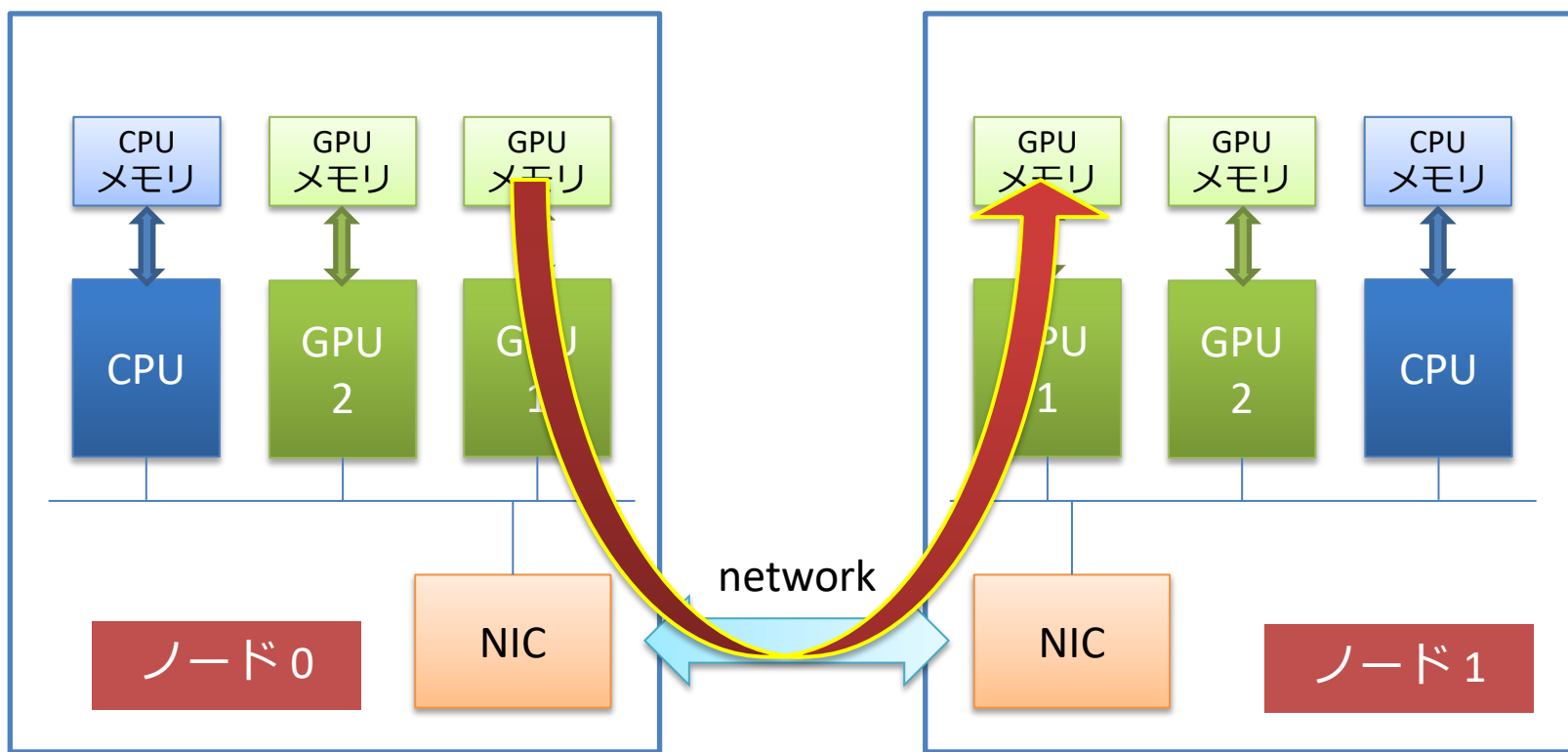
CUDA対応MPIライブラリ (CUDA-aware MPI)

- 非対応のMPIに比べてMPIの記述（引数等）に違いは無し
- CUDA向けの実装が内部的に施されている
 - ▶ OpenMPI, MVAPICHなどが対応
- コンパイル時や実行時に指定が必要な場合もある
 - ▶ 使い方はそれぞれの環境のマニュアルやサポートに問い合わせ
- NVIDIA HPC SDK
 - ▶ CUDA-aware MPIとしてOpenMPIが同梱済み
 - ▶ 特別な指定無しにCUDA対応の機能込みでコンパイルできる

GPUDirect RDMA転送

異なるノード間でGPUメモリが直接転送/やりとり可能

科学大TSUBAME 4.0, 東大Aquariusなどで採用



ホスト↔デバイス間転送コストを削減

方法2に追加可能な高速化の機能

- コード記述はCUDA対応MPI利用と同じ
- ハード・ソフトに利用条件あり
- 実行時またはコンパイル時に環境変数又はオプションを指定してON/OFFする
 - ▶ センターの管理者に確認を推奨
- 性能面でのメリットが期待出来る

複数GPUとOpenACCのまとめ

■ MPI通信関数がOpenACCのデータ領域**外**の場合

- ▶ OpenACCの構文を追加する必要なし
- ▶ ホスト上のポインタがMPIライブラリに正しく渡される

■ MPI通信関数がOpenACCのデータ領域**内**の場合

- ▶ update指示文をMPI通信関数の前後に入れる
 - ▶ 通常のMPIライブラリで使用する
- ▶ host_data構文でデバイスポインタを渡す
 - ▶ CUDA対応MPIで使える手法
 - ▶ 計算機環境によってはGPUDirect RDMAの利用も検討
- ▶ ユニファイドメモリを使う
 - ▶ ディレクティブ追加不要、MPIライブラリへの受け渡しは自動で処理

参考: MPSについて (1/2)

■ 次のようなGPU搭載の計算機の例を考える (典型例)

- ▶ CPU側: 1ノードに複数のプロセッサコア (例: 96コア, 72コア)
- ▶ GPU側: 1ノードに複数基のGPU → 典型的にはCPU側のプロセッサコアの数よりは少ない

■ MPIを使った複数GPUの利用

- ▶ 基本: 1 MPIタスクとGPU 1基を結びつける (binding)
- ▶ 質問: CPU側が複数プロセッサコアの場合、余ったコアをどうするか?
 - ▶ 対応0: 気にしない。CPU側のプロセッサコアを余らせておく (簡単)。
 - ▶ 対応1: CPU側のGPUと結びついていないコアで別の処理をさせる。
→ コード開発が大変だが、効果的なリソース利用に繋がる可能性がある。
 - ▶ 例: OpenMPのスレッド並列, 例: MPIでcommunicatorを分ける
 - ▶ 対応2: 基本をやめる。ノードあたりに複数のMPIタスクを動かす (簡単)。
 - ▶ 重要な注意: GPU 1基に複数のプロセスが結びつく可能性 (oversubscription)
→ 素朴に適用すると、大幅な性能劣化を引き起こす可能性がある

参考: MPSについて (2/2)

■ NVIDIA Multi-Process Service (MPS)の利用

- ▶ <https://docs.nvidia.com/deploy/mps/index.html>
- ▶ 対応2に対する一つのアプローチ → 複数プロセスで特定のGPUをシェアする場合でも、性能劣化を回避できる (かもしれない)
 - ▶ GPUリソースを効率的に管理する仕組み
 - ▶ `nvidia-cuda-mps-control`コマンドを利用する
 - ▶ 使用しているGPU搭載計算機で利用できるかどうかは確認が必要
- ▶ 事例
 - ▶ GROMACS (計測例あり): <https://developer.nvidia.com/blog/maximizing-gromacs-throughput-with-multiple-simulations-per-gpu-using-mps-and-mig/>
 - ▶ LAMMPS (ドキュメントのみ): https://docs.lammps.org/Speed_kokkos.html

Tipsの紹介

Tipsの紹介

1. コンパイル時のプリプロセッサマクロ
2. 簡易出力情報（環境変数の設定による）
3. GPUのハードウェア情報の確認

_OPENACC

■ OpenACCコンパイルを示す、プリプロセッサのマクロ

- ▶ OpenACC向けコードを選択してコンパイルするなどを利用
 - ▶ OpenMPディレクティブと混在させる時に利用できる
- ▶ GPU特有の処理、エラー処理、結果チェック、OpenACC用のI/O、OpenACCのAPI、OpenACC向け通信処理やデバイスidの選択など

```
#ifdef _OPENACC
```

-accオプション設定時のみ
有効な領域

```
#endif
```

accディレクティブの無視だけでは異なるプロセッサへの対応が出来ないような場合に活用

簡易出力情報（環境変数の設定）

■ NVCOMPILER_ACC_TIME=1（コストのサマリの出力）

▶ カーネル実行の情報、データ転送のコストなど

```
Accelerator Kernel Timing data  
/home/tyamagishi/000openACC/GPUseminar/src/jacobi.c
```

```
main NVIDIA devicenum=0
```

```
time(us): 26,810,166
```

```
59: data region reached 200 times
```

データ転送

```
59: data copyin transfers: 200
```

```
device time(us): total=9,077,044 max=45,649 min=45,302 avg=45,385
```

データ転送のコスト

```
70: data copyout transfers: 100
```

```
device time(us): total=4,064,125 max=40,715 min=40,634 avg=40,641
```

```
60: compute region reached 100 times
```

```
60: data copyin transfers: 100
```

```
device time(us): total=779 max=22 min=6 avg=7
```

```
63: kernel launched 100 times
```

カーネル実行

```
grid: [256x128] block: [32x4]
```

```
device time(us): total=284,493 max=2,859 min=2,832 avg=2,844
```

```
elapsed time(us): total=286,654 max=2,908 min=2,852 avg=2,866
```

カーネル実行のコスト

```
63: reduction kernel launched 100 times
```

```
grid: [1] block: [256]
```

```
device time(us): total=5,958 max=61 min=59 avg=59
```

```
elapsed time(us): total=7,865 max=89 min=76 avg=78
```

```
63: data copyout transfers: 100
```

```
device time(us): total=1,291 max=24 min=12 avg=12
```

Copyin, copyoutが発生した
行番号を確認できる

```
70: data region reached 200 times
```

```
...
```

※nsysとの同時利
用は不可
（正しい性能情報
が得られない）

簡易出力情報（環境変数の設定）

■ NVCOMPILER_ACC_NOTIFY=3（時系列データの出力）

- ▶ 1：カーネル、2：データ転送、3：1と2両方（組み合わせ可能）
- ▶ 4：Compute/Data領域のenter/exit、8：デバイスの非同期実行と同期処理、16：デバイスメモリの確保と解放

```
upload CUDA data  file=/home/tyamagishi/000openACC/GPUseminar/src/jacobi.c function=main line=59 device=0 threadid=1 variable=A
bytes=536870912
upload CUDA data  file=/home/tyamagishi/000openACC/GPUseminar/src/jacobi.c function=main line=59 device=0 threadid=1
variable=Anew bytes=536870912

...

launch CUDA kernel  file=/home/tyamagishi/000openACC/GPUseminar/src/jacobi.c function=main line=63 device=0 threadid=1
num_gangs=32768 num_workers=4 vector_length=32 grid=256x128 block=32x4 shared memory=2048
launch CUDA kernel  file=/home/tyamagishi/000openACC/GPUseminar/src/jacobi.c function=main line=63 device=0 threadid=1
num_gangs=1 num_workers=1 vector_length=256 grid=1 block=256 shared memory=2048

...

download CUDA data  file=/home/tyamagishi/000openACC/GPUseminar/src/jacobi.c function=main line=63 device=0 threadid=1 bytes=8
download CUDA data  file=/home/tyamagishi/000openACC/GPUseminar/src/jacobi.c function=main line=70 device=0 threadid=1
variable=Anew bytes=536870912
```

nvaccelinfoでGPUの仕様を確認

```
CUDA Driver Version:      11020
NVRM version:             NVIDIA UNIX x86_64 Kernel Module  460.73.01  Thu Apr  1 21:40:36 UTC 2021

Device Number:            0
Device Name:              Tesla V100-PCIE-32GB
Device Revision Number:   7.0
Global Memory Size:      34089730048
Number of Multiprocessors: 80
Concurrent Copy and Execution: Yes
Total Constant Memory:   65536
Total Shared Memory per Block: 49152
Registers per Block:     65536
Warp Size:               32
Maximum Threads per Block: 1024
Maximum Block Dimensions: 1024, 1024, 64
Maximum Grid Dimensions: 2147483647 x 65535 x 65535
Maximum Memory Pitch:    2147483647B
Texture Alignment:       512B
Clock Rate:              1380 MHz
Execution Timeout:       No
Integrated Device:       No
Can Map Host Memory:     Yes
Compute Mode:            default
Concurrent Kernels:      Yes
ECC Enabled:             Yes
Memory Clock Rate:       877 MHz
Memory Bus Width:        4096 bits
L2 Cache Size:          6291456 bytes
Max Threads Per SMP:     2048
Async Engines:           7
Unified Addressing:      Yes
Managed Memory:         Yes
Concurrent Managed Memory: Yes
Preemption Supported:    Yes
Cooperative Launch:      Yes
Multi-Device:           Yes
Default Target:          cc70
```

NVIDIA HPC SDKが利用できる環境で
nvaccelinfoコマンドを実行

HWの仕様一覧が表示される

発展にて最適化を行う際に有用な
情報を含む（blockの構成など）

基礎では以下の2つ抑えておけばOK

Managed Memory: Yes

Default Target: cc70
: OpenACCコンパイル時オプションに設定する

参考文献 (1/2)

1. OpenACC Getting Started Guide, <https://docs.nvidia.com/hpc-sdk/compiler/openacc-gs/index.html>
2. CUDA C++ Programming Guide, <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
3. HPC WORLD: HPC技術コラム, <https://hpcworld.jp/techcolumn/>
4. GDEP, GPUコラム「OpenACCではじめるGPUプログラミング」, <https://gdep-sol.co.jp/gpu-programming/>
5. Jeff Larkin, “Versatile OpenACC Interoperability Techniques” , <https://developer.nvidia.com/blog/3-versatile-openacc-interoperability-techniques/>
6. Open Hackathons Official Training Materials, <https://github.com/openhackathons-org>
7. Jiri Kraus, “Multi_GPU_Programming_with_MPI_and_OpenACC”, <https://github.com/jirikraus>

参考文献 (2/2)

8. Mark Harris, “Unified Memory for CUDA Beginners”, <https://developer.nvidia.com/blog/unified-memory-cuda-beginners/>
9. John Hubbard, Gonzalo Brito, Chirayu Garg, Nikolay Sakharnykh, and Fred Oh, “Simplifying GPU Application Development with Heterogeneous Memory Management”, <https://developer.nvidia.com/blog/simplifying-gpu-application-development-with-heterogeneous-memory-management/>
10. PCAST (Parallel Compiler Assisted Software Testing); <https://docs.nvidia.com/hpc-sdk/compiler/hpc-compilers-user-guide/index.html#pcast>
11. Detecting Divergence Using PCAST to Compare GPU to CPU Results; <https://developer.nvidia.com/blog/detecting-divergence-using-pcast-to-compare-gpu-to-cpu-results/>

謝辞

- 本資料 (サンプルコード, 計測) の一部において、東京科学大学のスーパーコンピュータTSUBAME4.0を利用しました。

Deep copyの対応

“複合的な”データの転送 (1/2)

■ 次のようなデータのデバイスへの転送を検討する

- ▶ C: 構造体, C++: クラス, Fortran: 派生型
 - ▶ 特徴: 複数のデータ (メンバー) から構成
- ▶ 変数名に加え, デバイスの計算で必要となる構成データも転送

```
typedef struct vector {  
    double x[32];  
    double y[32];  
    double z[32];  
} Vec3D;  
Vec3D s;
```

構造体の変数名を転送

```
#pragma acc data copy(s) \  
copyin(s.x[0:32], s.y[0:32]) copyout(s.z[0:32])  
{  
    #pragma acc kernels loop  
    for (int i = 0; i < 32; ++i) {  
        s.z[i] = s.x[i] + s.y[i];  
    }  
} /* acc data */
```

構成データも転送

C

```
type vector  
    real(8) :: x(32)  
    real(8) :: y(32)  
    real(8) :: z(32)  
end type  
type(vector) :: s
```

派生型の変数名を転送

```
!$ACC data copy(s) &  
!$ACC & copyin(s%x(1:32), s%y(1:32)) copyout(s%z(1:32))  
!$ACC kernels loop  
do i = 1, 32  
    s%z(i) = s%x(i) + s%y(i)  
end do  
!$ACC end kernels loop  
!$ACC end data
```

構成データも転送

Fortran

“複合的な”データの転送 (2/2)

■ “copy”(データ転送)の種類

- ▶ shallow copy: “複合的な”データの変数名のみの転送
- ▶ deep copy : 変数名に加え, 構成データ(メンバー)も転送する

■ OpenACCにおけるdeep copy

- ▶ 方針1: (コンパイラ任せにせず) デバイスで必要なデータを適切に転送する
 - ▶ classやmodule: コンストラクタや宣言時に転送するとよい
- ▶ 方針2: “複合的な”データを転送しないコーディングを目指す
 - ▶ 例: ポインタの扱いは難しい
- ▶ 方針3: ユニファイドメモリを使用 → 配慮は不要となる

手動deep copyの例: class [C++]

参考: Sec.2.5 in <https://docs.nvidia.com/hpc-sdk/compilers/openacc-gs/index.html>

```
template <typename T>
class myvec
{
public:
    T* v;
    myvec(const int ns_)
    {
        ns = ns_;
        v = (T *)malloc ( ns*sizeof(T) );
        #pragma acc enter data copyin(this[0:1]) \
        create(v[0:ns])
    }
    ~myvec()
    {
        #pragma acc exit data delete(v[0:ns], this[0:1])
        free( v );
    }

    void update_host()
    { // update指示文 with self節:ホスト側のデータを更新
        #pragma acc update self(v[0:ns])
    }
    void update_device()
    { // update指示文 with device節:デバイス側のデータを更新
        #pragma acc update device(v[0:ns])
    }
    // ...
};
```

コンストラクタで:

- 自分自身(thisポインタ)を転送
- メンバの配列をcreate節を利用してデバイス側で確保

デストラクタでデバイス側のメモリを全て解放

enter data構文とexit data構文:
data構文の機能を入力側 (enter)と出口側(exit)とに
分離した構文

```
{
    myvec<double> x(nsize);
    myvec<double> y(nsize);
    myvec<double> z(nsize);

    // set data in host
    for( int i =0; i< nsize; ++i ) {
        x[i] = 1.0;
        y[i] = 2.0;
        z[i] = 0.0;
    }

    // update data in device
    x.update_device();
    y.update_device();
    z.update_device();

    for (int itr = 0; itr < nrep; ++itr) {
        #pragma acc kernels loop independent \
        present(z,x,y) \
        present(z.v[0:nsize],x.v[0:nsize],y.v[0:nsize])
        for (int i = 0; i < nsize; ++i ){
            z.v[i] = x.v[i] + y.v[i];
        }
    }
    // update data in host
    z.update_host();
}
```

コンストラクタの呼び出しによりデバイスデータが確保

デバイスデータのlifetimeはブロックスコープ末端

手動deep copyの例: module [Fortran]

参考: Sec.2.9 in <https://docs.nvidia.com/hpc-sdk/compiler/openacc-gs/index.html>

enter data構文とexit data構文:
data構文の機能を入力側 (enter)
と出口側(exit)とに分離した構文

```
use mymod
implicit none
! ...
s%nsz = nsz
allocate( s%x(1:nsz) )
allocate( s%y(1:nsz) )
allocate( s%z(1:nsz) )
! ...
!$ACC enter data copyin(s%x(1:nsz),s%y(1:nsz),s%z(1:nsz))

do itr = 1, nitr
  !$ACC kernels loop present(s,s%x(1:nsz),s%y(1:nsz),s%z(1:nsz))
  do i = 1, nsz
    s%z(i) = s%x(i) + s%y(i)
  end do
end do

!$ACC exit data copyout(s%z(1:nsz))
```

```
module mymod
  implicit none
  integer,parameter :: SP = kind(1.0)
  integer,parameter :: DP = selected_real_kind(2*precision(1.0_SP))

  type vector
    integer :: nsz
    real(kind=DP),allocatable,dimension(:) :: x, y, z
  end type

  type(vector) :: s
  !$ACC declare copyin(s)
end module mymod
```

派生データ型の宣言時にデバイス側
に変数名を転送 (declare構文)

構成データを転送 (copyin)

構成データを転送 (copyout)

本資料は、構成・文章・画像などの全てにおいて著作権法上の保護を受けています。以下の条件で本資料の利用を許可します。

- 本資料の利用に際し、著作者への連絡は不要ですが、著作者のクレジット（当機構の名称）は明示して下さい。
- 本資料の一部あるいは全部について、転載、複製及び再配布を許可します。ただし、本資料を改変した場合は除きます。
- 本資料を販売するなどの営利目的での利用は禁じます。教育目的等で利用することは産・学・官を問わず許可します。

※上記はクリエイティブ・コモンズ「表示-非営利-改変禁止 4.0 国際（CC BY-NC-ND 4.0）」相当です。

- 本資料に記載された内容などは、予告なく変更される場合があります。
- 本資料に起因して使用者に直接または間接的損害が生じても、著作者はいかなる責任も負わないものとします。

なお、本資料の旧版についても上記と同様の条件で利用することを許可します。

ご不明の点があれば下記のヘルプデスクにお問い合わせ下さい。

ヘルプデスク： helpdesk[-at-]hpci-office.jp（[-at-]を@にしてください）

